3.2 Convolutions for images

- now that we understand how convolutional layers work in theory, we are ready to see how they work in practice
- building on our motivation of convolutional neural networks as efficient architectures for exploring structure in image data, we stick with images as our running example
- recall that, strictly speaking, convolutional layers are a misnomer, since the operations they express are more accurately described as cross-correlations
- based on our descriptions of convolutional layers, in such a layer, an input tensor and a kernel tensor are combined to produce an output tensor through a cross-correlation operation

3.2 Convolutions for images

- let us ignore channels for now, and see how this works with two-dimensional data and hidden representations
- in Figure 3, the input is a two-dimensional tensor with a height of 3 and width of 3
- we mark the shape of the tensor as 3×3 or $(3, 3)$
- the height and width of the kernel are both 2
- the shape of the *kernel window* (or *convolution window*) is given by the height and width of the kernel (here, it is 2×2)

3.2 Convolutions for images

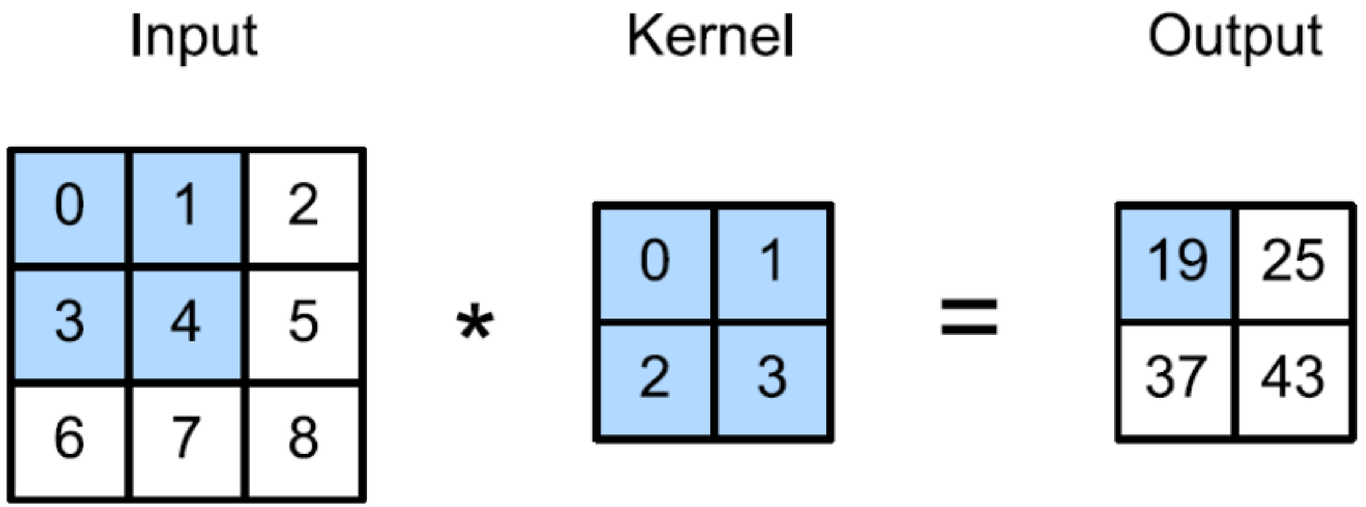

Figure 3: Two-dimensional cross-correlation operation. The shaded portions are the first output element, as well as the input and kernel tensor elements used for the output computation: $0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3 = 19$.

3.2 Convolutions for images

- in the two-dimensional cross-correlation operation, we begin with the convolution window positioned at the upper-left corner of the input tensor, and slide it across the input tensor, both from left to right and top to bottom
- when the convolution window slides to a certain position, the input subtensor contained in that window and the kernel tensor are multiplied element-wise, and the resulting tensor is summed up, yielding a single scalar value
- this result gives the value of the output tensor at the corresponding location
- here, the output tensor has a height of 2 and width of 2, and the four elements are derived from the two-dimensional cross-correlation operation:

$$
\begin{aligned}
0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3 &= 19, \\
1 \times 0 + 2 \times 1 + 4 \times 2 + 5 \times 3 &= 25, \\
3 \times 0 + 4 \times 1 + 6 \times 2 + 7 \times 3 &= 37, \\
4 \times 0 + 5 \times 1 + 7 \times 2 + 8 \times 3 &= 43.
\end{aligned}
$$

3.2 Convolutions for images

- note that, along each axis, the output size is slightly smaller than the input size
- because the kernel has width and height greater than one, we can only properly compute the cross-correlation for locations where the kernel fits wholly within the image, the output size is given by the input size $n_h \times n_w$ minus the size of the convolution kernel $k_h \times k_w$ via:

$$(n_h - k_h + 1) \times (n_w - k_w + 1).$$

- this is the case, since we need enough space to “shift” the convolution kernel across the image
- later, we will see how to keep the size unchanged by padding the image with zeros around its boundary, so that there is enough space to shift the kernel

3.2 Convolutions for images

- a convolutional layer cross-correlates the input and kernel, and adds a scalar bias to produce an output
- the two parameters of a convolutional layer are the *kernel* and the scalar *bias*
- when training models based on convolutional layers, we typically initialize the kernels randomly, just as we would with a fully-connected layer
- in $h \times w$ convolution or a $h \times w$ convolution kernel, the height and width of the convolution kernel are h and w, respectively
- we also refer to a convolutional layer with a $h \times w$ convolution kernel simply as a $h \times w$ convolutional layer

3.2 Convolutions for images

- let us take a moment to discuss a simple application of a convolutional layer: detecting the edge of an object in an image by finding the location of the pixel change
- in this case, the kernel can be constructed as $[1, -1]$, i.e., it has a height of 1 and a width of 2
- designing an edge detector by finite differences $[1, -1]$ is good, if we know this is precisely what we are looking for
- however, as we look at larger kernels, and consider successive layers of convolutions, it might be impossible to specify precisely what each filter should be doing, manually
- thus, we need to *learn* these kernels

3.2 Convolutions for images

- recall our observation of the correspondence between the cross-correlation and convolution operations
- here, let us continue to consider two-dimensional convolutional layers
- what if such layers perform strict convolution operations as defined in (2), instead of cross-correlations?
- in order to obtain the output of the strict *convolution* operation, we only need to flip the two-dimensional kernel tensor both horizontally and vertically, and then perform the *cross-correlation* operation with the input tensor
- it is noteworthy that, since kernels are learned from data in deep learning, the outputs of convolutional layers remain unaffected, no matter if such layers perform either the strict convolution operations, or the cross-correlation operations

3.2 Convolutions for images

- to illustrate this, suppose that a convolutional layer performs *cross-correlation* and learns the kernel in Figure 3, which is denoted as the matrix **K** here
- assuming that other conditions remain unchanged, when this layer performs strict *convolution* instead, the learned kernel $\mathbf{K}'$ will be the same as **K** after $\mathbf{K}'$ is flipped both horizontally and vertically
- that is to say, when the convolutional layer performs strict *convolution* for the input in Figure 3 and $\mathbf{K}'$, the same output in Figure 3 (cross-correlation of the input and **K**) will be obtained
- in keeping with standard terminology with deep learning literature, we will continue to refer to the cross-correlation operation as a convolution even though, strictly-speaking, it is slightly different
- besides, we use the term *element* to refer to an entry (or component) of any tensor representing a layer representation or a convolution kernel

3.2 Convolutions for images

- as described before, the convolutional layer output in Figure 3 is sometimes called a *feature map*, as it can be regarded as the learned representations (features) in the spatial dimensions (e.g., width and height) to the subsequent layer
- in CNNs, for any element x of some layer, its *receptive field* refers to all the elements (from all the previous layers) that may affect the calculation of x during the forward propagation
- note that the receptive field may be larger than the actual size of the input

3.2 Convolutions for images

- let us continue to use Figure 3 to explain the receptive field
- given the 2×2 convolution kernel, the *receptive field* of the shaded output element (of value 19) is the four elements in the shaded portion of the input
- now, let us denote the 2×2 output as **Y** and consider a deeper CNN with an additional 2×2 convolutional layer that takes **Y** as its input, outputting a single element z
- in this case, the receptive field of z on **Y** includes all the four elements of **Y**, while the receptive field on the input includes all the nine input elements
- thus, when any element in a feature map needs a larger receptive field to detect input features over a broader area, we can build a deeper network